

Jetson Camera Detection Service — IT

Deployment Guide

Audience: IT / systems administrators deploying and operating this service. No Python or machine-learning knowledge is assumed.

What this service does: it runs on a small NVIDIA computer (a "Jetson") installed at a site, connects to that site's IP cameras over the local network, analyses the video on the Jetson itself to detect people and vehicles, and makes those detections available over HTTP to the central management platform.

Video is analysed locally. Only detection results — not video — are sent to the platform by this service.

1. Before you start

1.1 Identify the board

This is the most important step. The two boards need different software and are not interchangeable.

On the device, open a terminal and run:

```
cat /proc/device-tree/model; echo
```

Output contains	Board	Git branch to use	Setup script
Orin Nano	Jetson Orin Nano (current)	orin-nano	setup_orin.sh
Nano without Orin	Original Jetson Nano (legacy)	main	setup_nano.sh

The reason is technical but absolute: the two boards ship different versions of NVIDIA's TensorRT inference library (version 10 on the Orin, version 8 on the original Nano), and the code that drives it differs between them. **Running the wrong branch on a board will fail at startup.** The setup scripts detect this mismatch and refuse to continue, rather than installing something broken.

1.2 Requirements

	Orin Nano	Original Nano
OS image	JetPack 6.x / 7.x	JetPack 4.6.x
RAM	8 GB	4 GB (shared with GPU)
Free disk	12 GB	4 GB
Power	Supplied PSU	Barrel-jack PSU, not micro-USB
Network	Wired Ethernet to the camera LAN	Same
Internet	Required during setup only	Same
Install time	45–75 min (mostly unattended download)	60–90 min

Also required: `sudo` rights on the device, and the RTSP address plus credentials for each camera.

Original Nano, 10 W mode: maximum performance requires the barrel-jack power supply. On micro-USB the board can brown out and reboot under load.

1.3 Capacity — how many cameras per device

Plan capacity before you deploy. Exceeding it does not degrade gracefully; it drops frames and the detection overlay falls behind the live video.

Board	Recommended	Frame rate per camera	Total frames/sec
Orin Nano (8 GB)	8–10 cameras	12 fps	~120
Original Nano (4 GB)	2 cameras	12 fps	~24
	4 cameras	6–8 fps	~28
	10 cameras (not advised)	2.5–3 fps	~30

The original Nano's limit is processing speed, not memory. It sustains only **25–40 frames per second** in

total, across all cameras combined — one 128-core GPU, no multi-camera batching on this board, and image preparation that does not overlap with GPU work. Memory is comfortable to roughly 40 cameras; the processor is the wall long before that.

Divide the total by your camera count to get the per-camera frame rate. Ten cameras at 12 fps would require 120 frames/sec — **three to five times what this board can deliver**. Asking for it does not fail loudly; it drops frames and the detection overlay drifts behind the live video.

For more than 4 cameras, specify an Orin Nano.

2. Installation

2.1 Copy the software to the device

```
git clone https://github.com/peshalnepal/1886N0ENTRYBACKEND.git
cd 1886N0ENTRYBACKEND
```

Then select the branch for your board (from the table in 1.1):

```
git checkout orin-nano      # Jetson Orin Nano
# or
git checkout main           # original Jetson Nano
```

2.2 Run the setup script

```
cd tensort/deployment
chmod +x *.sh
./setup_orin.sh             # or ./setup_nano.sh
```

Enter the `sudo` password when prompted. The script then runs unattended for 45–90 minutes and prints progress through eight numbered steps.

The script is safe to re-run. If it is interrupted by a power cut, a dropped SSH session, or a closed window, run the same command again — it resumes and skips completed work.

2.3 What the script does

Step	Action
1	Preflight — checks board type, RAM, disk, swap and network. Stops immediately if the board is wrong or disk is short.
2	Installs the NVIDIA software stack and video plugins (the large download).

Step	Action
------	--------

- | | |
|---|---|
| 3 | Verifies CUDA, TensorRT and OpenCV, including the GStreamer check in 5.2. |
| 4 | Enables maximum performance mode. |
| 5 | Creates an isolated Python environment and installs dependencies. |
| 6 | Builds the AI model for this specific device. |
| 7 | Writes the configuration file <code>.env</code> . |
| 8 | Installs the background service, starts it, and confirms it is healthy. |

Step 6 — why it takes time and cannot be skipped. The AI model is compiled into a file optimised for the exact chip and driver version in the device in front of you. A model file copied from another machine **will not load**. This is why installation must run on each device rather than being imaged once.

Optional settings, for re-runs:

Setting	Effect
---------	--------

<code>SKIP_APT=1</code>	Skip the large download (already completed)
-------------------------	---

<code>SKIP_ENGINE=1</code>	Skip rebuilding the AI model
----------------------------	------------------------------

<code>SKIP_SERVICE=1</code>	Skip service install and health check
-----------------------------	---------------------------------------

<code>MAX_BATCH=10</code>	Orin only: size the model for this many cameras
---------------------------	---

<code>PORT=8080</code>	Change the service port
------------------------	-------------------------

Example: `SKIP_APT=1 ./setup_orin.sh`

2.4 Confirm the installation

The script ends with a health check. To confirm at any later time, from any machine on the same network:

```
curl http://<jetson-ip>:8080/health
```

A response containing "pipeline_ready": true means the service is working.

3. Adding cameras

In normal operation the central platform adds cameras automatically. The commands below are for commissioning and fault-finding.

Each camera needs a **source address** and a **name** (camera_uuid).

```
curl -X POST http://<jetson-ip>:8080/cameras \
  -H "Content-Type: application/json" \
  -d '{"source_url":"rtsp://user:pass@192.168.1.50:554/stream1","camera_uuid":"front-door"}'
```

Supported address formats:

Format	Example
RTSP (most IP cameras)	rtsp://user:pass@192.168.1.50:554/stream1
WebRTC / WHEP	whep://host/stream/whep
HLS / MJPEG	http://host/stream/index.m3u8
RTMP	rtmp://host/app/streamkey
SRT	srt://host:8890?streamid=...

Other useful commands:

```
curl http://<jetson-ip>:8080/cameras # list cameras
curl http://<jetson-ip>:8080/cameras/front-door/latest # current detections
curl http://<jetson-ip>:8080/cameras/front-door/snapshot.jpg --output snap.jpg
curl -X DELETE http://<jetson-ip>:8080/cameras/front-door # remove a camera
```

Cameras are stored on the device and restored automatically after a reboot. Add each camera once.

4. Operating the service

The service runs under systemd as **jetson-cameras** and starts automatically at power-on.

```

sudo systemctl status jetson-cameras    # is it running
sudo systemctl restart jetson-cameras  # restart
sudo systemctl stop jetson-cameras      # stop
sudo journalctl -u jetson-cameras -f    # live log (Ctrl+C exits the log only)
tegrastats                             # live CPU / GPU / memory usage

```

4.1 Health monitoring

curl http://<jetson-ip>:8080/health returns the service state. Monitor these:

Field	Meaning	Action if wrong
pipeline_ready	Service is operational	If false, see 5.1
infer_dropped	Frames discarded, unable to keep up	Must stay near 0. If climbing, reduce cameras or frame rate (5.4)
channel_count	Cameras currently connected	Should equal your camera count
max_batch	Cameras processable per GPU pass	If 1 on an Orin, batching is off — see 5.3
mem_total_mb	Detected RAM	Sanity check for the right board

A reasonable alerting rule: **alert if /health is unreachable, if pipeline_ready is false, or if infer_dropped rises continuously over 15 minutes.**

5. Troubleshooting

5.1 pipeline_ready is false / the service will not start

```

sudo journalctl -u jetson-cameras -n 80 --no-pager

```

Log message	Cause	Fix
DET_ENGINE not found	AI model missing	Re-run the setup script
FileNotFoundError: ...engine	Model not built on this device	Re-run the setup script
execute_async_v3 / num_io_tensors errors	Wrong branch for this board	git checkout main on an original Nano;

Log message	Cause	Fix
		re-run setup
Killed / out of memory	Insufficient memory during model build	Add swap (5.5), re-run

5.2 Cameras never connect, or images are blank

The most common root cause is a Python package shadowing the system video libraries. Check:

```
cd ~/1886N0ENTRYBACKEND/tensort
source .venv_trt/bin/activate
python3 -c "import cv2,re;print('GStreamer:', bool(re.search(r'GStreamer:\s+YES',
cv2.getBuildInformation())))"
```

If this prints `False`, hardware video decoding cannot work. Fix:

```
pip uninstall -y opencv-python opencv-python-headless
sudo systemctl restart jetson-cameras
```

Otherwise verify the camera address, credentials, and that the Jetson can reach the camera (ping `<camera-ip>`). Test the stream directly:

```
gst-launch-1.0 rtspsrc location="rtsp://user:pass@<camera-ip>:554/stream1" ! fakesink
```

5.3 Detection is slow, or the overlay lags behind the video

Check `infer_dropped` in `/health`. If it is rising, the device is being asked for more than it can deliver. **Raising the frame rate makes this worse, not better.** Reduce the camera count or the frame rate (5.4).

On an Orin, also check `max_batch` in `/health`. If it reports `1`, the AI model was built without multi-camera batching and throughput is far below capacity. Rebuild it:

```
cd ~/1886N0ENTRYBACKEND/tensort/deployment
SKIP_APT=1 MAX_BATCH=10 ./setup_orin.sh
```

If it still reports `1`, the supplied model file lacks a variable batch dimension and must be re-exported by the development team.

5.4 Reducing load

Edit `tensort/.env`, then `sudo systemctl restart jetson-cameras`:

Setting	Effect
DEFAULT_SAMPLE_FPS	Frames analysed per camera per second. The main lever. Orin: 12. Nano: 2.
EMIT_EMPTY_DETECTION S	false reduces network traffic substantially
DEFAULT_RESIZE_W/H	Smaller frames use less memory

5.5 Adding swap (original Nano)

If the model build is killed for lack of memory:

```
sudo fallocate -l 4G /swapfile && sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

5.6 Starting over

```
sudo systemctl stop jetson-cameras
rm ~/1886NOENTRYBACKEND/tensorrt/jetson_cameras.db
sudo systemctl start jetson-cameras
```

This clears all registered cameras. Configuration and the AI model are kept.

6. Security

Read this before exposing a device beyond the local network.

The service has **no authentication**. Any host that can reach port 8080 can list, add and delete cameras, and retrieve snapshot images. The central platform can be configured to send an API key, but **this service does not check it**.

Required controls:

1. **Never expose port 8080 to the internet.** No port-forwarding, no DMZ.
2. Place Jetsons on a **dedicated camera VLAN**, isolated from general traffic.
3. Restrict port 8080 to the management platform's address only:

```
sudo ufw allow from <platform-ip> to any port 8080 proto tcp
sudo ufw deny 8080
sudo ufw enable
```

1. For remote sites, connect over a **VPN**, not a public port.
2. Camera passwords are stored in clear text in the local database (`jetson_cameras.db`) — physical security of the device matters.
3. Change the default `ubuntu/nvidia` OS password on first login.

7. Reference

7.1 Files on the device

Path	Purpose
<code>tensort/.env</code>	Configuration (edit, then restart)
<code>tensort/models/*.engine</code>	AI model, built for this device — never copy between devices
<code>tensort/jetson_cameras.db</code>	Registered cameras
<code>tensort/deployment/</code>	Setup scripts
<code>/etc/systemd/system/jetson-cameras.service</code>	Service definition

7.2 HTTP endpoints (port 8080)

Method	Path	Purpose
GET	<code>/health</code>	Service status and statistics
GET	<code>/cameras</code>	List cameras
POST	<code>/cameras</code>	Add a camera
PATCH	<code>/cameras/<id></code>	Change camera settings
DELETE	<code>/cameras/<id></code>	Remove a camera
GET	<code>/cameras/<id>/latest</code>	Most recent detections

Method	Path	Purpose
GET	/cameras/<id>/ snapshot.jpg	Current image
GET	/cameras/detections/ stream	Continuous detection feed

All paths also work with an `/api` prefix.

7.3 Known limitations

Limitation	Impact
No authentication on port 8080	Network controls are mandatory (section 6)
Built-in web server is single-process	Above ~10 cameras, or many simultaneous streams, throughput is limited
AI model is device-specific	Every device must run setup individually; no golden image
Original Nano is 2–3 cameras at 2 fps	Specify Orin Nano for larger sites
<code>MAX_CAMERAS</code> in <code>.env</code>	Not enforced by the software — capacity is an operational limit

8. Escalation

Collect this before escalating to the development team:

```
cat /proc/device-tree/model; echo
git -C ~/1886NOENTRYBACKEND rev-parse --abbrev-ref HEAD
curl -s http://localhost:8080/health
sudo journalctl -u jetson-cameras -n 200 --no-pager
free -h; df -h
```

Include the board model, the branch, the health output, and the log.